

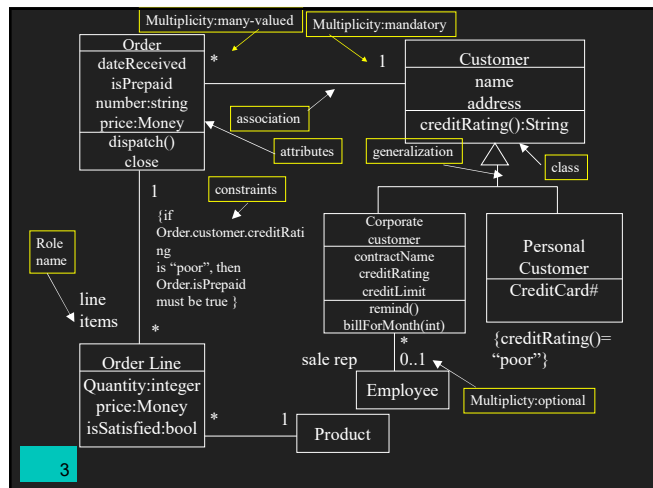
UML Class Diagram

1

Class Diagram

- A class diagram describes the classes and the association (static relationship) between these classes (描述系統中物件的 type(class)，以及這些class 的靜態關連.)
 - association (for example, a customer may rent a number of videos)
 - subtypes (a nurse is a kind of person)

2

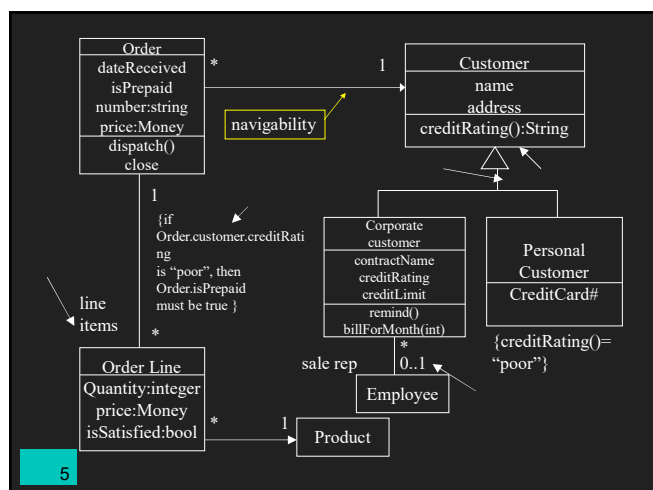


3

navigability

- See next fig
- In specification model, an Order has a responsibility to tell you which Customer it is for, but not conversely
- In implementation diagram, this would indicate that Order contains a pointer to Customer
- UML 2.x remove this feature

4



5

Generalization

- Generalization (In OOP term, is inheritance)
- see personal and corporate customers
- Using polymorphism, the corporate customer may respond to certain commands differently from another customer

6

Operations

- Most obviously correspond to the methods on a class
- UML syntax is
visibility name (parameter list); return-type-expression {property string}

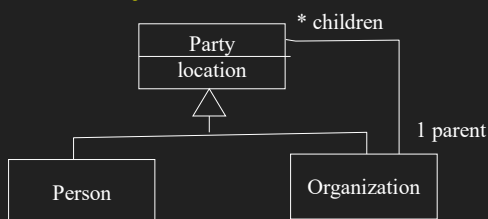
7

Class Diagrams: Advanced Concepts

8

Object Diagram

- A snapshot of the objects in a system at a point in time
- Often called instance diagram
- Let's see the class diagram first



9

Party::dosomething()

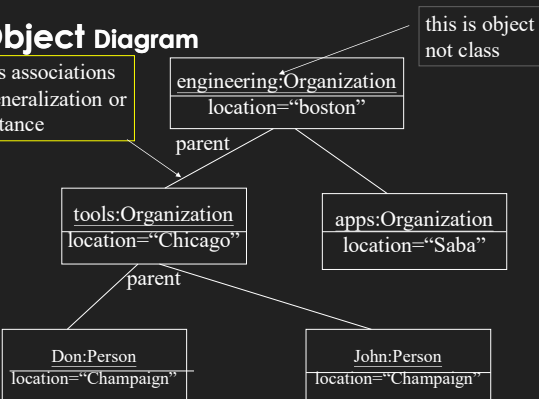
```

Class party {
    location :
    abstract int computeSalary
}
Class person : party {
    computeSalary() {
        return his personal wages ;
    }
}
Class organization : party {
    vector<party*> children ;
    computeSalary () {
        for each o in children
            total += o.computeSalary() ;
        }
    }
}
  
```

10

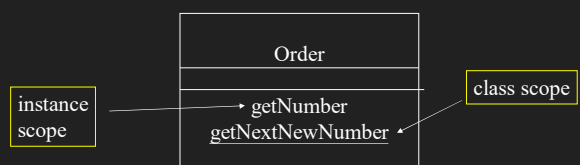
Object Diagram

This is associations
not generalization or
inheritance



11

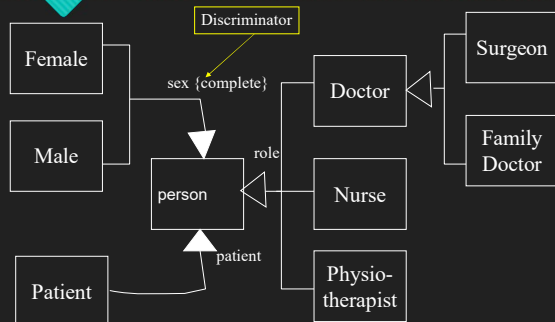
Class Scope Operations and Attributes



Class scope is equivalent to static members in C++

12

Multiple and Dynamic Classification



13

13

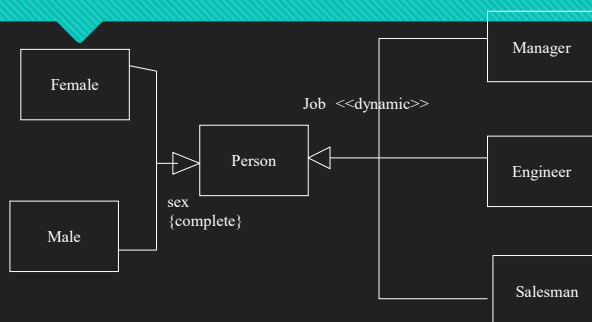
Multiple Classification

- Don't confuse with multiple inheritance.
- You should make it clear which combinations are legal by using a discriminator

14

14

Dynamic Classification



15

15

Dynamic Classification

- allow object to change type within the subtyping structure
- useful for conceptual modeling
- How to implement it? See Fowler 1997 video tape rental example (state design pattern)

16

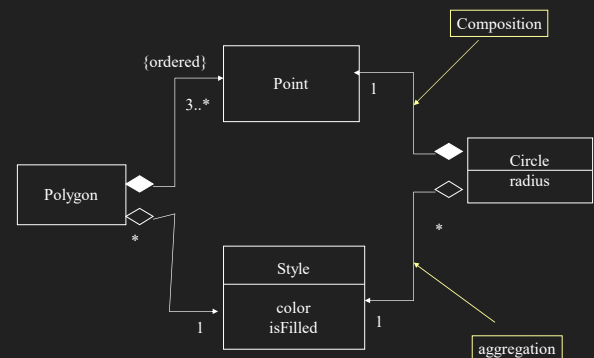
16

Aggregation and Composition

- Aggregation is the part-of relationship
- difficult things - what is the difference between aggregation and association?
 - It is vague and inconsistent
 - Anyway, UML decide to include aggregation and a stronger variety of aggregation called composition
- Basically an aggregation/composition relation is still an association. They are used to emphasize the type of association

17

17



18

18

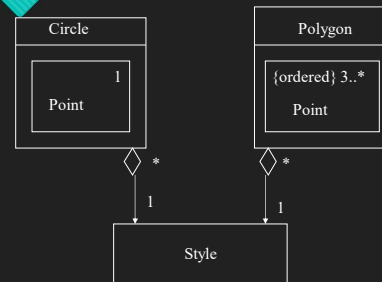
Composition

- With composition, the part object may belong to only one whole; further, the parts are usually expected to live and die with the whole
- deletion of the whole is considered to cascade to the part
- In previous graph, deleting a polygon would caused its associated *Points* to be deleted, but not the associated *Style*.

19

19

Alternative Notation for Composition



20

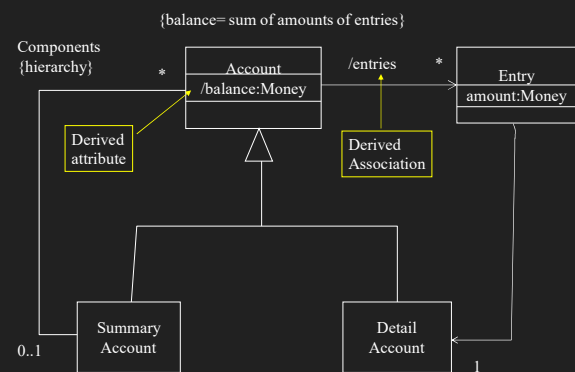
20

Derived Associations and Attributes

- Derived Associations and derived attributes can be calculated from other associations and attributes.
 - for example, an age attribute of a Person can be derived if you know that person's birthday.

21

21



22

22

Note

- Entry objects are attached to detail Accounts
- The balance of an Account is calculated as the sum of Entry accounts
- A Summary Account's entries are the entries of its components determined recursively

23

23

Interfaces and Abstract Classes

- Programming language that use a single construct, the class, which contains both *interface* and *implementation*.
- When you subclass, you inherit both.
- A pure interface, as in Java, is a class with no implementation and, therefore, has operation declarations but no method bodies and no fields.

24

24

For example

```
interface Stack {
    boolean Push(Object);
    Object Pop();
}
```

Somewhere in initialization

```
Stack S = new MyStack();
```

Class MyStack implements Stack

Somewhere in the code

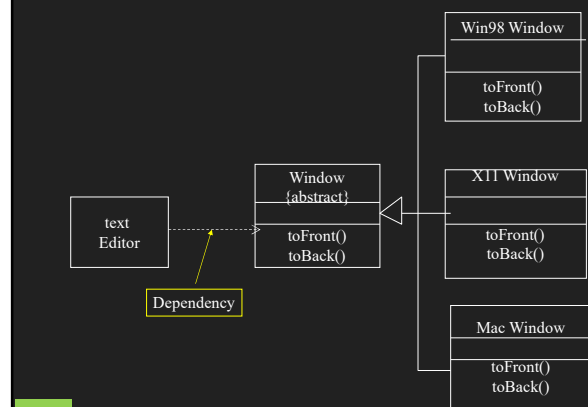
```
boolean Push(Object) {
    .....
}

Object Pop() {
    .....
    return xxxx;
}
```

S.push(a);
S.push(b);
S.pop(a);
S.pop(b);

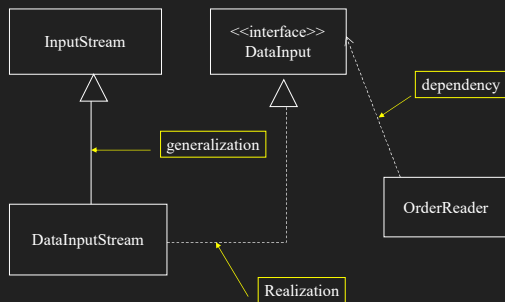
25

25



26

26



27

27

Abstract Class and Interface

- two are similar
- abstract class allows you to add implementation of some of the methods
- an interface forces you to defer definition of all methods

28

28

Classification and Generalization

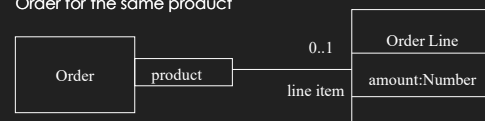
- People often talk subtyping as the "is a" relationship
- beware of that way of thinking
- for example
 - Shep is a border Collie
 - A border Collie is a Dog
 - Dogs are animals
 - A border Collie is a Breed (品種)
 - Dog is a Species (物種)
- if you try combination 2 and 5 "A border Collie is a Species". This is not good
- The reasons?
 - some are classification and some are generalization
- Generalization is transitive and classification is not
- "Dog are a kind of animals" is better

29

29

Qualified Associations

- equivalent to associative arrays, maps, and dictionaries
- an example, there maybe one Order Line for each instance of Product
- Conceptually, you cannot have two Order Lines within an Order for the same product



30

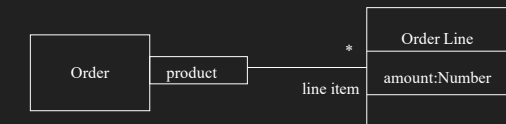
30

Qualified association would imply an interface like

```
class Order {
    public OrderLine getLineItem (Product aProduct);;
    public void addLineItem (Number amount, Product
forProduct);
}
```

31

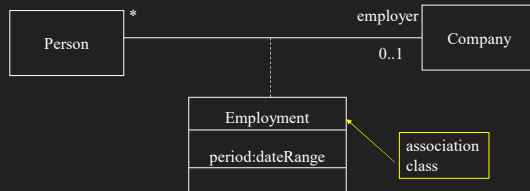
- You can have multiple OrderLines per Product but access to the Line Items is still indexed by Product
- use an associative array or similar data structure to hold the order lines



32

Association Class

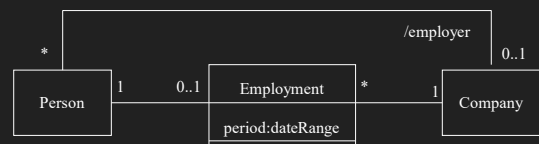
- Association class allow you to add attributes, operations, and other features to association



33

33

- A person may work for a single company
- We need to keep information about the period of time that each employee works for each Company
- You can redraw: make Employment a full class in its own right



34

34

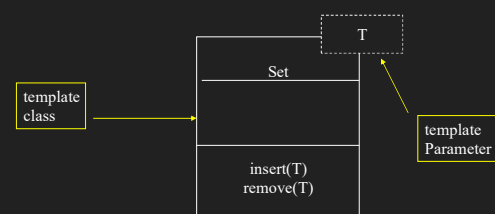
Parameterized Class

- Several language, noticeably C++, have the notion of a parameterized class or template
- ex
class set <T> {
void insert (T newElement);
void remove(T anElement);
...
Set <Employee> employSet ;

35

35

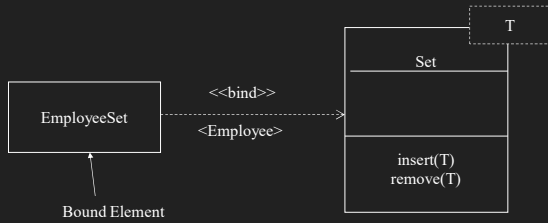
A define a parameterized class in UML



36

36

A use of a parameterized class



37

37

Visibility

- C++
 - A public member is visible anywhere in the program and may be called by any object within the system
 - A private member may be used only by the class that defines it
 - A protected member may be used only by (a) the class that defines it or (b) a subclass of that class
- In Java
 - a protected member may be accessed by subclasses but also by any other class in the same package as the owning class
- C++
 - one C++ method or class can be made a friend of a class. A friend has complete access to all members of a class

38

38